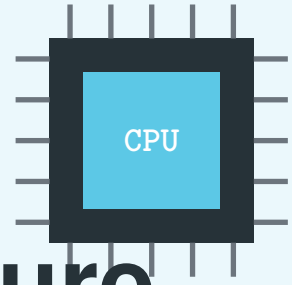
Course UEF31

Computer Architecture



Architecture des ordinateurs

- Chapter 1: General organization of the central unit
- Chapter 2: Inside the processor
- Chapter 3: Case study, the Intel 8086 processor
- Chapter 4: Case study, 32-bit and 64-bit processors
- Chapter 5: Modern processor architectures

Teacher: Dr. Benabderrezak Youcef

Coefficient: 3 **Credits:** 5

Assessment: exam 60%, continuous assessment 40%

How to use this course

This course is written for you, even if this is the first time you look inside a computer. We start with simple ideas. Step by step, we go up to real programs in assembly language. At the end, we understand how the processors in our phones and laptops work.

The coloured boxes

On every page you will find coloured boxes. Each colour has a job:

Definition

A new important word. We also give its French name, because you will meet it in the official syllabus and in many books.

Everyday analogy

A difficult idea, explained with something you know from daily life: a kitchen, a school, a bus...

Step by step

A calculation or an execution, done one small step at a time. We never skip a step.

Watch out

A common mistake, or a small detail that changes the result.

Did you know?

A fun extra fact. You do not need it for the exam, but it helps you understand.

The programs in this course

- Every program is complete, and comes with a table **"What does each line do?"**.
- The 8086 programs are written for the **EMU8086** emulator. The 64-bit programs are written for **NASM** on **Linux**.
- All the programs **were really run**. What you see in the black screen is the real output, and the register tables come from the real execution.

Watch out

Do not only read! Copy each program into EMU8086, run it one instruction at a time (the *Single step* button), and watch the registers change. This is how you learn this course.

Contents

How to use this course **1**

1 General Organization of the Central Unit **4**

1.1 What is a computer? . . . 4

1.1.1 Hardware and software . . . 5

1.1.2 The layers of a computer . . . 5

1.1.3 How does a program run? . . . 5

1.2 A short history of computers . . . 5

1.3 Quick reminder: how does a computer count? . . . 6

1.3.1 Memory size units . . . 8

1.4 The main parts of a computer . . . 8

1.5 The two basic models: Von Neumann and Harvard . . . 9

1.5.1 The Von Neumann model . . . 9

1.5.2 The Harvard model . . . 10

1.5.3 Comparison and uses . . . 10

1.6 The processor: role and parts . . . 11

1.6.1 The clock and the speed of the processor . . . 11

1.7 Memory . . . 11

1.7.1 What does memory look like inside? . . . 12

1.7.2 Only two operations: read and write . . . 12

1.7.3 Types of memory . . . 12

1.7.4 Calculating the size of a memory . . . 13

1.8 Buses . . . 14

1.8.1 Bus width . . . 15

1.9 Exercises . . . 15

Chapter 1

General Organization of the Central Unit

Summary: what will we learn in this chapter?

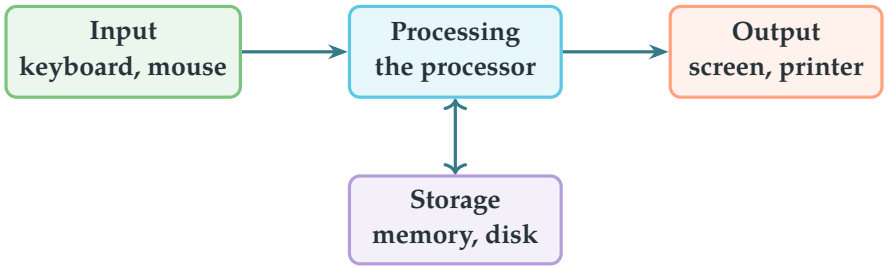
- What a computer is, and how it changed through history.
- The main parts: the processor, the memory, the input/output devices.
- The Von Neumann and Harvard models, and the difference between them.
- The types of memory, and how to calculate a memory size.
- The three buses: the data bus, the address bus, the control bus.

1.1 What is a computer?

Definition

A **computer** (Fr. *ordinateur*) is an electronic machine. It receives **data**, processes it with a **program** (a list of instructions), and gives **results**. It can also store data and results.

Everything a computer does fits in four verbs: it **receives**, it **processes**, it **stores**, it **outputs**.



Let us take a simple example: you use the calculator app on your computer to compute 25×4.

1. **Input**: you type 25, then $\times$, then 4 on the keyboard. Each key press becomes a number code that goes into the computer.
2. **Temporary storage**: the computer keeps the two numbers in memory, because it needs them soon.
3. **Processing**: the processor reads the two numbers, runs the multiply instruction, and gets 100.
4. **Output**: the computer sends the result to the screen, and you see it.

Every program you use, even a complex one (a game, a browser, a photo app), is in fact millions of these small steps, done very fast.

Everyday analogy

A computer is like a **restaurant kitchen**. The ingredients (vegetables, meat) are the **data**. The recipe is the **program**. The cook is the **processor**. The fridge is the **memory**. The finished dish is the **result**. The cook does not decide alone: he follows the recipe step by step.

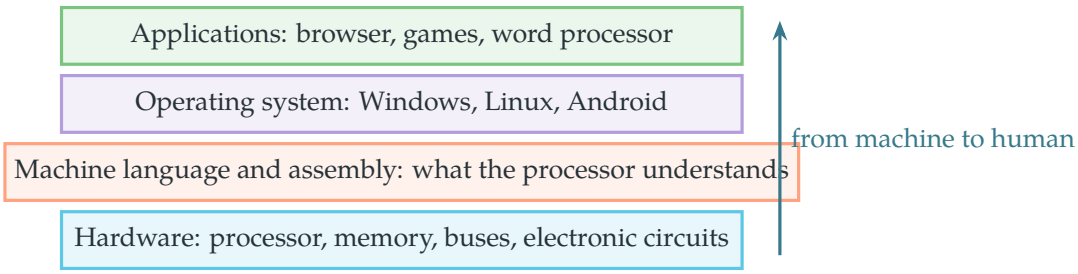
1.1.1 Hardware and software

- **Hardware** (Fr. *matériel*): everything you can touch: the processor, the memory, the screen, the wires.
- **Software** (Fr. *logiciel*): the programs that tell the hardware what to do: the operating system, the browser, games.

In this course we study **the hardware from the inside**, and we see how it runs programs.

1.1.2 The layers of a computer

We can see a computer as layers, one on top of the other. Each layer uses the services of the layer below it, without knowing all its details:



In this course we work on the **two lowest layers**: the hardware, and the assembly language that talks to it directly.

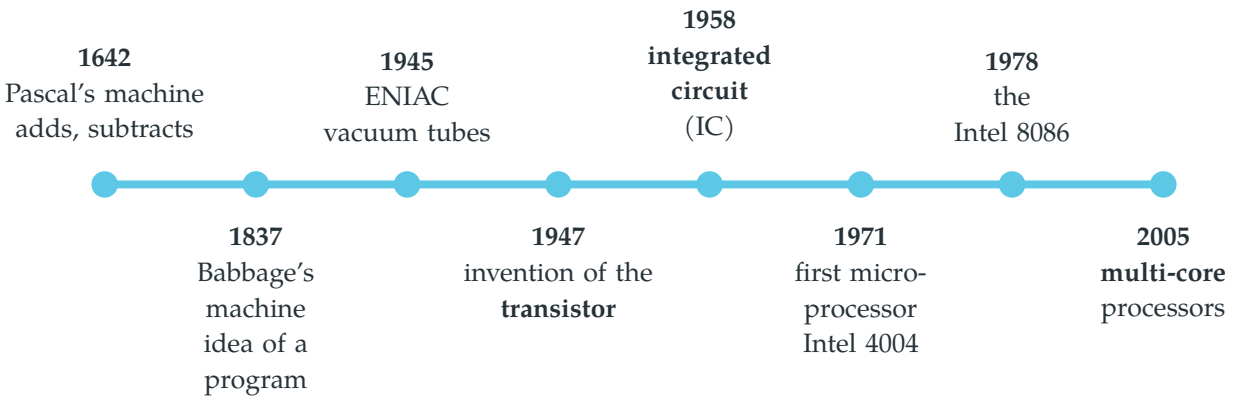
1.1.3 How does a program run?

When you double-click on a program icon, these steps happen:

1. The program is stored on the **disk** (SSD or HDD) as a file of zeros and ones.
2. The operating system **copies it into the main memory** (RAM), because the disk is very slow and the processor does not read from it directly.
3. The processor reads the instructions from memory **one after the other** and runs them.
4. When you close the program, its space in memory is freed for other programs.

1.2 A short history of computers

The computer did not appear in one day. These are the main steps:



Generation	Period	Main technology	Example
1st	1945–1955	vacuum tubes	ENIAC (30 tons!)
2nd	1955–1965	transistor	IBM 7090
3rd	1965–1980	integrated circuits	IBM 360
4th	1980–today	microprocessor (VLSI)	personal computer, phone

Each new generation brought a technology that made the parts **smaller**, **faster** and **less hungry for energy**:

- **Vacuum tubes** were big, got very hot, and burned out all the time. Some had to be changed almost every day.
- A **transistor** does the same job (an electric switch that opens and closes), but it is hundreds of times smaller and does not burn out.
- An **integrated circuit** puts thousands of transistors on one small piece of silicon, instead of connecting them with wires.
- A **microprocessor** puts the **whole** processor on one chip. This gave us the personal computer, then the smartphone.

Definition

Moore's law (Fr. *loi de Moore*): in 1965, Gordon Moore (a founder of Intel) noticed that the number of transistors on a chip **doubles about every two years**. This stayed true for more than fifty years: from 2 300 transistors in the 4004 to tens of billions today.

Did you know?

ENIAC filled a whole room and used 150 kilowatts. Your phone is millions of times more powerful and runs on a small battery. The reason: transistors became tiny, so one processor now holds billions of them.

1.3 Quick reminder: how does a computer count?

A computer only knows two electric states: **current flows** or **current does not flow**. We write them 1 and 0.

Definition

- A **bit** (Fr. *bit*): the smallest unit of information. Its value is 0 or 1.
- A **byte** (Fr. *octet*): a group of **8 bits**.
- A **word** (Fr. *mot*): the number of bits the processor handles at once (16 bits in the 8086).

We write numbers in three systems: decimal (base 10), binary (base 2), and hexadecimal (base 16, its digits are 0 to 9, then A to F).

Decimal	Binary	Hex	Decimal	Binary	Hex
0	0000	0	8	1000	8
1	0001	1	9	1001	9
2	0010	2	10	1010	A
3	0011	3	11	1011	B
4	0100	4	12	1100	C
5	0101	5	13	1101	D
6	0110	6	14	1110	E
7	0111	7	15	1111	F

Step by step: convert $2F_H$ to binary and to decimal

1. Each hex digit is **4 bits**: $2 = 0010$ and $F = 1111$.
2. Put them side by side: $2F_H = 0010\,1111_2$.
3. To decimal: $2 \times 16 + 15 = 32 + 15 = 47$.

Step by step: convert the binary number 1011 to decimal

Each bit has a "weight", a power of 2. We start on the right with $2^0 = 1$:

Bit	1	0	1	1
Weight	$2^3 = 8$	$2^2 = 4$	$2^1 = 2$	$2^0 = 1$
Bit × weight	8	0	2	1

We add: $8 + 0 + 2 + 1 = 11$. So $1011_2 = 11$.

Step by step: convert the decimal number 13 to binary (divide by 2 again and again)

1. $13 \div 2 = 6$, remainder **1**.
2. $6 \div 2 = 3$, remainder **0**.
3. $3 \div 2 = 1$, remainder **1**.
4. $1 \div 2 = 0$, remainder **1**. We stop, because the quotient is 0.
5. Read the remainders **from the last to the first**: 1101_2. Check: $8 + 4 + 0 + 1 = 13$.

Everyday analogy

Why use hexadecimal? Because binary numbers are long and tiring to read: $1010\,1111\,0011\,0110_2$. Hexadecimal packs every 4 bits into one digit, so the number becomes $AF36_H$. It is the "short form" programmers use to write binary quickly, like writing "Dr." instead of "Doctor".

Watch out

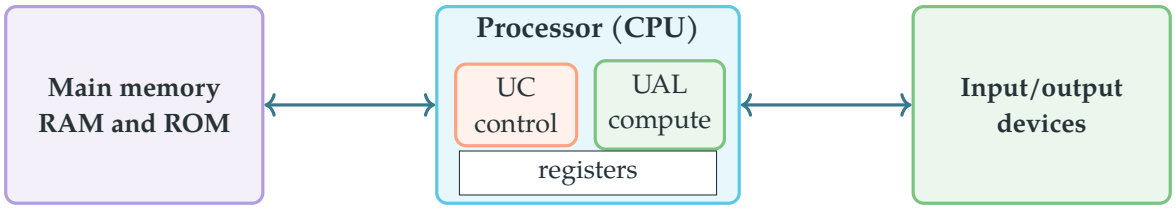
In assembly language we write `h` after a hex number (`2Fh`), and `b` after a binary number (`00101111b`). A number without a letter is decimal.

1.3.1 Memory size units

Unit	Symbol	Value
kilobyte	KB	$2^{10} = 1024$ bytes
megabyte	MB	$2^{20} = 1\,048\,576$ bytes
gigabyte	GB	2^{30} bytes (about one billion)
terabyte	TB	2^{40} bytes

1.4 The main parts of a computer

Inside every computer, from the biggest server to the smallest smartwatch, we find three main parts:



1. The **processor** (Fr. *processeur*) (CPU): the "brain" of the computer. It reads the instructions and runs them.
2. The **main memory** (Fr. *mémoire centrale*): it keeps the program that is running now, and its data.
3. The **input/output devices** (Fr. *périphériques d'E/S*): they let the computer talk with the outside world.

Step by step: the trip of a letter from the keyboard to the screen

You press the key **A** in a text editor:

1. The keyboard (**input**) sends the code of the key to the computer.
2. The processor is interrupted (it stops its work for a moment), reads the code, and stores it in **memory**, inside the text you are writing.
3. The text editor asks the processor to "draw" the letter: the processor computes the light dots that make the letter.
4. These dots go to the graphics card, then to the screen (**output**).

All this happens in a tiny part of a second, so the letter seems to appear at once.

Definition

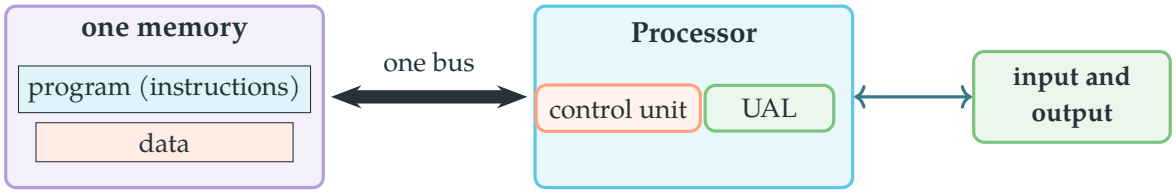
The **central unit** (Fr. *unité centrale*): the processor plus the main memory. This is the part that really runs the programs.

1.5 The two basic models: Von Neumann and Harvard

1.5.1 The Von Neumann model

In 1945, the scientist John von Neumann proposed a simple and brilliant idea: we keep the **program** and the **data** together in **the same memory**. Before that, changing the program meant re-connecting wires by hand!

In Von Neumann's model, a computer has **five units**: input, output, memory, the arithmetic and logic unit, and the control unit. The key idea is the **stored program** (Fr. *programme enregistré*): instructions are numbers, just like data, so we can keep them in the same memory, and read and change them the same way.



- **Advantage**: simple and flexible. To change the program, we just load another program into memory.
- **Drawback**: one bus for instructions and data, so we cannot read an instruction and a data item at the same moment. This traffic jam is called the **Von Neumann bottleneck** (Fr. *goulot de Von Neumann*).

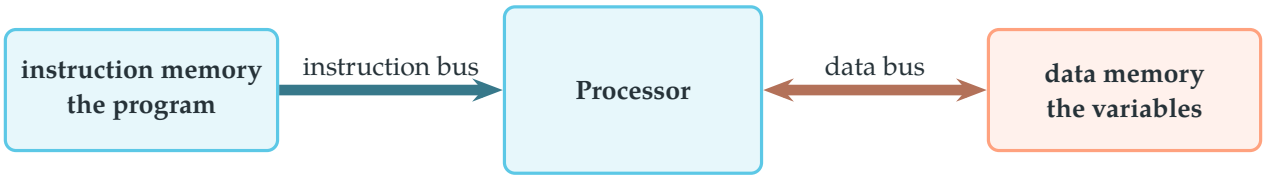
Everyday analogy

Imagine a library with **one narrow door**. The students who bring back books (the data) and the students who come for instructions (the program) all use the same door, one by one. Even if the

staff inside is very fast, the door decides the speed. This is the bottleneck.

1.5.2 The Harvard model

In this model we use **two separate memories**: one for instructions and one for data. Each one has its own bus.



- **Advantage**: faster, because the processor reads the next instruction and a data item at the same time.
- **Drawback**: more complex and more expensive, and the size of each memory is fixed.

Everyday analogy

In the Harvard library there are **two doors**: one for instructions and one for data. The two queues move at the same time, so people get in faster. But two doors cost more, and if one side is full we cannot use the free space of the other side.

1.5.3 Comparison and uses

	Von Neumann	Harvard
Memory	one memory for program and data	two separate memories
Buses	one shared bus	two independent buses
Speed	slower (bottleneck)	faster (two reads at once)
Complexity	simple and cheap	more complex, more expensive
Used in	personal computers, servers	microcontrollers (Arduino), signal processors (DSP)

Did you know?

The processors in your computer mix the two models. The main memory (RAM) is shared (Von Neumann), but inside the processor there are two small, fast, separate memories: one for instructions and one for data (Harvard). We call this **modified Harvard**. We come back to it in Chapter 5.

1.6 The processor: role and parts

Definition

The **processor** (Fr. *processeur, UCT*) (CPU): the circuit that runs the instructions of a program, one after the other.

The processor has three main parts (we study them in detail in Chapter 2):

Part	Its job	Analogy
control unit (UC)	reads the instruction, understands it, gives orders to the other parts	the orchestra conductor
arithmetic and logic unit (UAL)	does arithmetic ($+$, $-$) and logic (AND, OR)	the calculator
registers	very small and very fast memory inside the processor	the scrap paper on the desk

1.6.1 The clock and the speed of the processor

The processor works to the rhythm of a **clock** (Fr. *horloge*) that sends regular pulses. At each pulse the processor moves one small step forward in its work.

Definition

The **frequency** f: the number of pulses per second, measured in hertz (Hz). The **period** T: the length of one pulse, and $T = \frac{1}{f}$.

Step by step: a processor at 3 GHz

1. $f = 3\ \text{GHz} = 3 \times 10^9$ pulses per second.
2. $T = \frac{1}{3 \times 10^9} \approx 0.33 \times 10^{-9}$ second, so about 0.33 nanosecond.
3. In the time you blink (about 0.1 second), this processor does 300 million cycles!

Watch out

The frequency alone does not decide the speed of a processor. What matters is the **number of instructions done per second**. A 3 GHz processor that needs 4 cycles per instruction runs $\frac{3\times10^9}{4} = 750$ million instructions per second. A 2 GHz processor that runs 2 instructions per cycle runs 4×10^9 instructions per second: it is faster, even with a lower frequency.

1.7 Memory

1.7.1 What does memory look like inside?

Imagine memory as a **big building full of boxes**. Each box has a number called the **address**, and each box holds **one byte**.

address	content
0000	3A
0001	FF
0002	07
0003	41
0004	00

Address 0003 holds 41_H.
The address never changes; the content can change.

Everyday analogy

The address is like the **house number** in a street, and the content is like **the person who lives there**. The house number is fixed; the people living there can change.

1.7.2 Only two operations: read and write

- **Read**: the processor sends an address, and the memory gives back the content (the content does not change).
- **Write**: the processor sends an address and a value, and the memory stores the value instead of the old one.

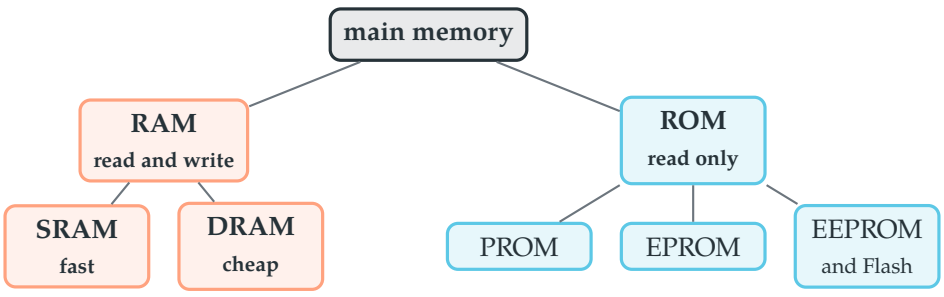
Example

Box 0002 holds 07_H. The processor runs "write 55_H at 0002". After that, the box holds 55_H, and 07_H is lost for ever. So we must be careful where we write: writing in the wrong place erases important data.

Definition

Random Access Memory (Fr. *mémoire à accès aléatoire*) (RAM): "random access" means we can reach **any box directly**, in the same time, without going through the boxes before it. This is the opposite of an old cassette tape, which you must wind until you reach the song.

1.7.3 Types of memory



Type	Description	When power is off	Used for
SRAM	transistors only, very fast and expensive	erased	cache memory inside the processor
DRAM	a tiny capacitor per bit, must be "refreshed" all the time	erased	main memory (RAM sticks)
ROM	written once in the factory	kept	old boot programs
EPROM	erased with ultraviolet light, then written again	kept	prototypes
EEPROM / Flash	erased electrically	kept	BIOS/UEFI, USB keys, SSD

Everyday analogy

SRAM is like a light switch: it stays in its position (on or off) as long as there is power. **DRAM** is like a glass with a small hole: the water (the electric charge) leaks slowly, so we must fill it again thousands of times per second. This is called **refresh**. But the glass is much smaller than the switch, so we can put billions of them on a cheap memory stick.

Did you know?

Why does a computer need ROM or Flash? When you switch it on, the RAM is completely empty. The processor needs a first program to start with: the boot program (BIOS or UEFI), kept in a memory that is not erased. This program checks the hardware, then finds the operating system on the disk and loads it into RAM.

Definition

A **volatile** (Fr. *volatile*) memory loses its content when the power is off (RAM). A **non-volatile** (Fr. *non volatile*) memory keeps it (ROM, Flash).

1.7.4 Calculating the size of a memory

If the processor has n address lines (wires), it can write 2^n different addresses.

Definition

number of boxes $= 2^n$ and size $= 2^n \times$ size of one box

Step by step: the memory size of the 8086

The 8086 has a **20-bit** address bus, and each box holds one byte.

1. Number of addresses: 2^{20}.
2. $2^{20} = 2^{10} \times 2^{10} = 1024 \times 1024 = 1\,048\,576$ addresses.
3. Size $= 2^{20} \times 1$ byte $= 1$ megabyte (1 MB).

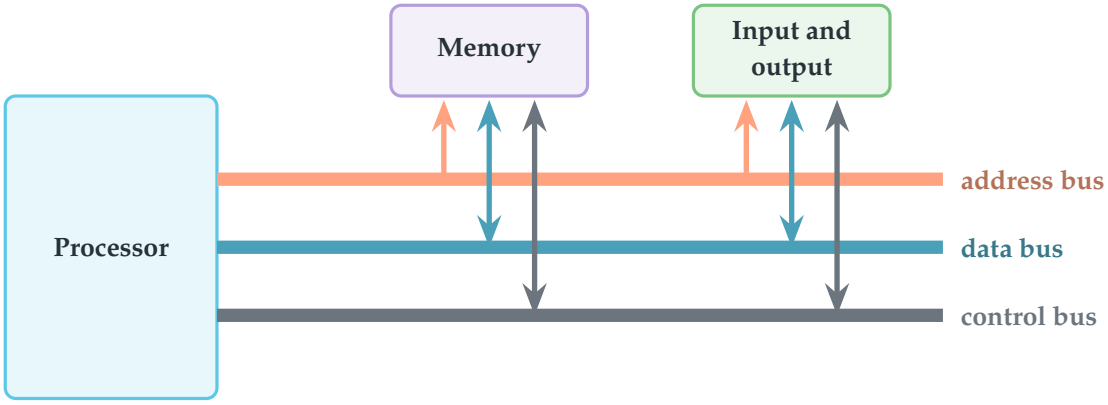
Step by step: how many address lines for a 64 KB memory?

1. $64\ \text{KB} = 64 \times 1024 = 2^6 \times 2^{10} = 2^{16}$ bytes.
2. So $2^n = 2^{16}$, and $n = 16$ lines.

1.8 Buses

Definition

A **bus** (Fr. *bus*): a group of parallel wires that carries information between the processor, the memory and the input/output devices. Each wire carries one bit at a time.



Bus	Direction	What does it carry?
address bus	from the processor only	the number of the box the processor wants to read or write
data bus	both ways	the value that is read or written
control bus	both ways	signals such as read (RD), write (WR), clock, interrupt

Everyday analogy

Imagine you ask for a parcel at the post office. The **address** tells the clerk which shelf you mean (address bus). The **parcel** itself is the data (data bus). The words "give me" or "take this" are the control signal (read or write).

Step by step: the processor reads box 0003

1. The processor puts the address 0003 on the **address bus**.
2. It sends a **read** signal (RD) on the **control bus**.
3. The memory puts the content of the box (41_H) on the **data bus**.
4. The processor takes the value 41_H and keeps it in one of its registers.

1.8.1 Bus width

- The **width of the address bus** decides the **maximum memory size**: n lines give 2^n addresses.
- The **width of the data bus** decides **how many bits move at once**: a 16-bit bus moves two bytes at a time.

Step by step: moving 1000 bytes with different buses

1. An **8-bit** data bus moves one byte at a time: we need 1000 transfers.
2. A **16-bit** bus moves two bytes: $1000 \div 2 = 500$ transfers.
3. A **64-bit** bus moves 8 bytes: $1000 \div 8 = 125$ transfers only.

The wider the bus, the faster the data moves, just as more cars pass on an 8-lane motorway than on a one-lane road.

Processor	Data bus	Address bus	Max memory
Intel 8086	16 bits	20 bits	$2^{20} = 1$ MB
Intel 80386	32 bits	32 bits	$2^{32} = 4$ GB
modern x86-64	64 bits	48 bits (in practice)	$2^{48} = 256$ TB

1.9 Exercises

Exercise 1.1

Convert these numbers: (a) $3C_H$ to decimal and binary. (b) The decimal number 200 to hexadecimal.

Solution

(a) $3C_H = 3 \times 16 + 12 = 60$, and in binary $0011\,1100_2$.
(b) $200 = 12 \times 16 + 8$, and 12 is written C, so $200 = C8_H$.

Exercise 1.2

A processor has a 24-bit address bus, and each memory box holds one byte. What is the maximum memory size?

Solution

$2^{24} = 2^4 \times 2^{20} = 16 \times 1$ MB $= 16$ MB.

Exercise 1.3

We want a 4 GB memory. How many lines does the address bus need?

Solution

$4\,\text{GB} = 2^2 \times 2^{30} = 2^{32}$ bytes, so we need **32 lines**.

Exercise 1.4

True or false? Explain. (a) In the Von Neumann model, the program is kept in a different memory from the data. (b) The address bus carries information in both directions. (c) DRAM keeps its content after the computer is switched off.

Solution

(a) **False**: Von Neumann has one memory for program and data; separation is the Harvard model.
(b) **False**: only the processor sends addresses, so it goes one way.
(c) **False**: DRAM is volatile; it is erased when the power goes off.

Exercise 1.5

A processor runs at 2 GHz, and each instruction needs 4 cycles on average. How many instructions does it run per second?

Solution

Cycles per second $= 2 \times 10^9$. Instructions $= \dfrac{2 \times 10^9}{4} = 0.5 \times 10^9$, that is **500 million instructions per second**.